

11.SE.301 – Rekayasa Perangkat Lunak

Keamanan Perangkat Lunak I
Konsep & Ancaman



S-1

Teknik Informatika

Pokok Pembahasan

- Software Security Engineering
- Security Life-Cycle Model
- Secure Development Life-Cycle Activities
- Security Requirements Engineering
- Misuse and Abuse Cases and Attack Patterns
- Security Risk Analysis
- Threat Modeling, Prioritization, and Mitigation

Sumber: Chapter 18 Buku Pressman & Chapter 13 Buku Sommerville

Software Security Engineering

Pengantar

Keamanan Perangkat Lunak tidak hanya berbicara tentang adanya:

- Firewalls
- Strong passwords
- Encryption

Software Security Engineering

- **Keamanan perangkat lunak (software security)** adalah konsep, proses, dan praktik yang diterapkan untuk memastikan bahwa suatu perangkat lunak tetap berfungsi sebagaimana mestinya, serta terlindungi dari berbagai ancaman, serangan cyber, manipulasi, atau akses tidak sah.
- Secara sederhana, jika rekayasa perangkat lunak (software engineering) fokus pada bagaimana membuat aplikasi *bisa berjalan*, maka keamanan perangkat lunak fokus pada bagaimana memastikan aplikasi tersebut *tidak bisa dirusak atau disalahgunakan*.

CIA

Dalam dunia keamanan IT, termasuk perangkat lunak, fondasi dasarnya merujuk pada tiga aspek utama:

- **Confidentiality (Kerahasiaan):** Memastikan data sensitive di dalam aplikasi (seperti password, data pribadi, atau rekam medis) hanya bisa diakses oleh pihak yang memiliki hak akses resmi.
- **Integrity (Integritas):** Menjamin bahwa data atau kode perangkat lunak tidak dapat diubah, dimanipulasi, atau dirusak oleh pihak yang tidak berwenang selama proses penyimpanan atau pengiriman.
- **Availability (Ketersediaan):** Memastikan perangkat lunak dan datanya selalu siap digunakan oleh pengguna yang sah kapan pun dibutuhkan (terbebas dari serangan seperti *Denial of Service / DoS* yang bisa membuat aplikasi *crash*).

Secure SDLC

Keamanan perangkat lunak yang baik tidak dipasang di akhir setelah aplikasi selesai dibuat, melainkan diintegrasikan sejak awal proses pembuatan. Konsep ini dikenal sebagai **Secure Software Development Life Cycle (Secure SDLC)**:

- **Analisis Kebutuhan:** Menentukan regulasi privasi dan standar keamanan apa yang harus dipenuhi.
- **Desain:** Melakukan *Threat Modeling* (memetakan potensi ancaman sebelum kode ditulis).
- **Pengodean (Coding):** Menerapkan standar penulisan kode yang aman (*secure coding*) untuk menghindari celah klasik seperti SQL Injection atau Cross-Site Scripting (XSS).
- **Pengujian (Testing):** Melakukan audit keamanan, mulai dari pemindaian otomatis hingga uji penetrasi (*penetration testing*) secara manual.
- **Penyebaran & Pemeliharaan (Deployment & Maintenance):** Memantau aplikasi saat berjalan dan merilis pembaruan (*patch*) secara berkala jika ditemukan celah baru.

Mengapa Keamanan Perangkat Lunak Sangat Krusial?

- **Mencegah Kerugian Finansial & Hukum:** Kebocoran data dapat berujung pada denda regulasi yang besar serta biaya pemulihan sistem yang mahal.
- **Menjaga Reputasi:** Sekali sebuah aplikasi atau platform terkenal terkena *hack* dan data penggunaanya bocor, kepercayaan publik akan menurun drastis.
- **Melindungi Kekayaan Intelektual:** Mencegah pembajakan, pencurian kode sumber (*source code*), atau manipulasi logika bisnis aplikasi.

Contoh Ancaman Umum pada Perangkat Lunak

Beberapa celah keamanan yang paling sering mengeksploitasi perangkat lunak antara lain:

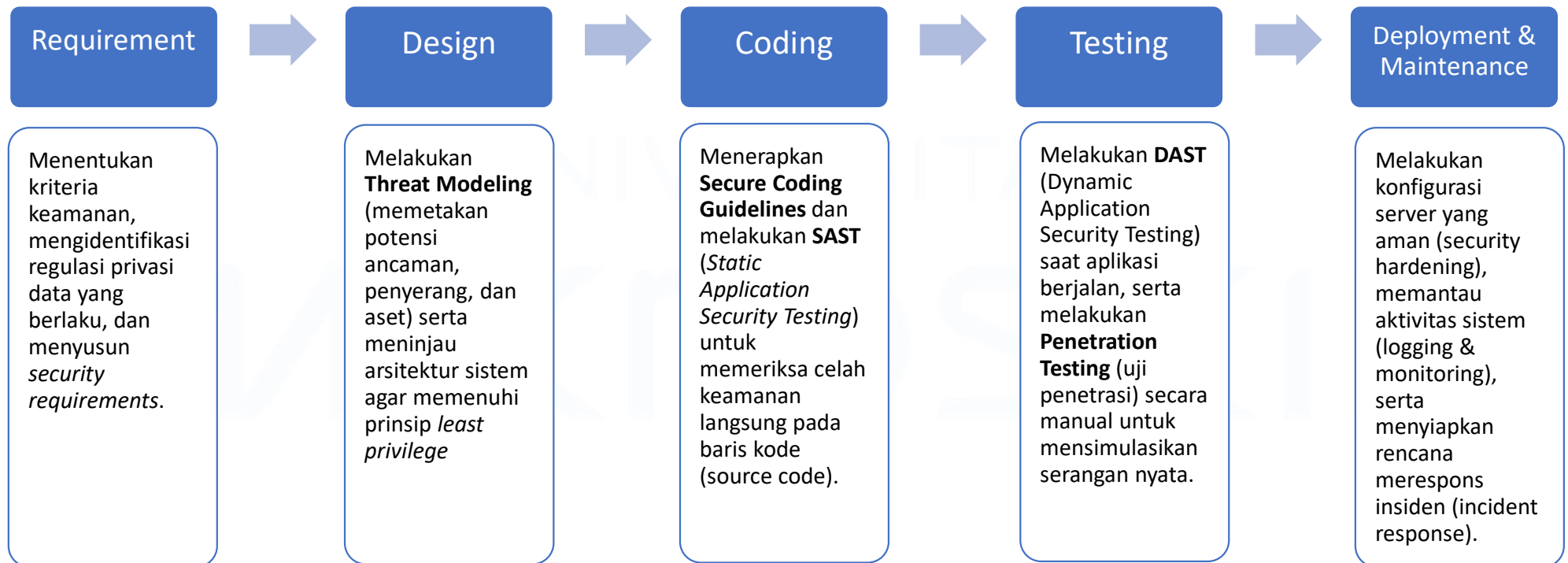
- **Injection Flaws:** Ketika penyerang memasukkan data berbahaya (misalnya perintah SQL) ke dalam kolom input aplikasi yang kemudian dieksekusi oleh server.
- **Broken Authentication:** Celah pada sistem login atau manajemen sesi yang membuat penyerang bisa memalsukan identitas pengguna lain.
- **Security Misconfiguration:** Konfigurasi server atau framework yang kurang ketat, seperti membiarkan pengaturan *default* atau mode *debugging* tetap aktif di masa produksi.

Security Life-Cycle Model

Security Life-Cycle Model

- **Security Life-Cycle Model** (atau sering disebut sebagai *Secure Software Development Life Cycle* / **Secure SDLC**) adalah sebuah kerangka kerja (framework) yang mengintegrasikan praktik, proses, dan pengujian keamanan ke dalam setiap tahapan pengembangan perangkat lunak dari awal hingga akhir.
- Jika model SDLC tradisional (seperti Waterfall atau Agile) hanya fokus pada fungsionalitas dan kecepatan rilis aplikasi, Security Life-Cycle Model memastikan bahwa **keamanan menjadi prioritas di setiap fase**, bukan sekadar tambalan (*patch*) yang dipasang setelah aplikasi selesai dibuat.

Security Life-Cycle Model



Manfaat Security Life-Cycle Model

- **Pengurangan Biaya Perbaikan:** Memperbaiki celah keamanan saat kode masih ditulis jauh lebih murah daripada memperbaiki sistem yang sudah jebol di tangan pengguna.
- **Kepatuhan Regulasi (Compliance):** Membantu organisasi memenuhi standar hukum dan regulasi industri (seperti OWASP, ISO 27001, atau regulasi perlindungan data pribadi).
- **Kesadaran Tim (Security Awareness):** Mendorong kolaborasi yang lebih erat antara tim pengembang (*Developers*), tim operasi (*Operations*), dan tim keamanan (*Security*), atau yang biasa dikenal dengan budaya **DevSecOps**.

Secure Development Life-Cycle Activities

Fase Inisiasi & Analisis Kebutuhan (Requirements)

Fase ini berfokus pada penentuan standar keamanan apa saja yang harus dipenuhi oleh aplikasi sejak awal.

- **Security Requirements Gathering:** Menentukan kebutuhan keamanan berdasarkan jenis aplikasi. Misalnya, jika aplikasi menangani transaksi uang, maka harus ada enkripsi standar industri.
- **Risk Assessment (Penilaian Risiko):** Menilai seberapa besar dampak kerugian jika aplikasi ini berhasil diserang, baik dari sisi bisnis, hukum, maupun reputasi.
- **Compliance & Privacy Review:** Memeriksa regulasi hukum perlindungan data yang harus dipatuhi (seperti UU PDP di Indonesia atau GDPR) terkait bagaimana data pengguna disimpan dan dikelola.

Fase Desain Arsitektur (Design)

Pada fase ini, tim fokus pada merancang sistem yang memiliki pertahanan kuat sebelum kode program mulai ditulis.

- **Threat Modeling (Pemodelan Ancaman):** Aktivitas terstruktur untuk mengidentifikasi potensi ancaman, agen penyerang (attacker), dan vektor serangan terhadap arsitektur aplikasi. Salah satu metode yang populer adalah **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).
- **Secure Architecture Review:** Meninjau desain sistem untuk memastikan diterapkannya prinsip *Least Privilege* (hak akses seminimal mungkin yang diperlukan) dan *Defense in Depth* (keamanan berlapis).

Fase Pengodean (Implementation / Coding)

Fase ini adalah saat para developer menulis kode program dengan mengikuti standar keamanan yang ketat.

- **Secure Coding Guidelines:** Menggunakan panduan penulisan kode yang aman untuk mencegah celah populer, seperti yang tercantum dalam dokumen **OWASP Top 10** (misalnya, validasi semua input untuk mencegah *SQL Injection*).
- **Static Application Security Testing (SAST):** Melakukan pemindaian otomatis terhadap kode sumber (*source code*) tanpa menjalankan aplikasinya. Tujuannya untuk menemukan pola kode yang rawan atau tidak aman secara *real-time*.
- **Software Composition Analysis (SCA):** Memeriksa pustaka (*library*) atau komponen *open-source* pihak ketiga yang digunakan dalam proyek, untuk memastikan tidak ada *dependency* yang memiliki celah keamanan bawaan (*vulnerability*).

Fase Pengujian (Verification / Testing)

Aktivitas di fase ini bertujuan untuk menguji ketahanan aplikasi secara dinamis saat sistem dijalankan.

- **Dynamic Application Security Testing (DAST):** Menguji aplikasi dari luar dalam kondisi berjalan (runtime). Alat DAST akan menyimulasikan serangan ke aplikasi untuk melihat bagaimana respons sistem terhadap input berbahaya.
- **Penetration Testing (Pentest):** Uji penetrasi manual yang dilakukan oleh ahli keamanan (*ethical hacker*) untuk mencoba membobol sistem secara legal, guna menemukan celah logika bisnis yang tidak bisa dideteksi oleh alat pemindai otomatis.

Fase Penyebaran & Operasional (Deployment & Maintenance)

Keamanan tidak berhenti setelah aplikasi dirilis; aktivitas pasca-rilis sangat krusial untuk menjaga sistem tetap aman di lingkungan nyata.

- **Environment Hardening:** Mengonfigurasi server, database, dan jaringan tempat aplikasi berjalan agar seaman mungkin (misalnya mematikan port yang tidak digunakan dan memperbarui sistem operasi).
- **Continuous Monitoring & Logging:** Memasang sistem pemantauan untuk mencatat aktivitas mencurigakan secara terus-menerus (*log management*), sehingga jika terjadi percobaan serangan, tim bisa mengetahuinya dengan cepat.
- **Incident Response Plan (IRP):** Menyusun prosedur standar tentang apa yang harus dilakukan oleh tim jika terjadi insiden kebocoran data atau serangan siber berhasil menembus sistem.

Aktivitas Secure SDLC

Fase SDLC	Requirements	Design	Coding	Testing	Deployment
Aktivitas	Security Requirements & Risk Assessment	Threat Modeling & Architecture Review	Secure Coding, SAST, & SCA	DAST & Penetration Testing	Hardening, Monitoring, & Incident Plan
Output	Dokumen Regulasi & Kriteria Keamanan	Diagram Ancaman (<i>Threat Model</i>)	Kode Bersih (<i>Clean & Secure Code</i>)	Laporan Celah Keamanan (<i>Vulnerability Report</i>)	Sistem yang Siap Beroperasi Aman

Security Requirements Engineering

Security Requirements Engineering

- **Security Requirements Engineering (SRE)** adalah fase spesifik dan krusial dalam rekayasa perangkat lunak yang berfokus pada **mengidentifikasi, menganalisis, dan menentukan kebutuhan keamanan** suatu sistem sejak tahap awal pengembangan.
- Jika rekayasa kebutuhan (*Requirements Engineering*) biasa berfokus pada apa yang *harus* dilakukan oleh sistem (fungsionalitas), maka SRE berfokus pada apa yang *tidak boleh* terjadi pada sistem dan bagaimana sistem mempertahankan diri dari penyalahgunaan.

Security Requirements Engineering

- Banyak kegagalan keamanan perangkat lunak bukan disebabkan oleh kesalahan penulisan kode (*coding error*), melainkan karena **salah desain** atau **kelalaian dalam mendefinisikan kebutuhan keamanan** sejak awal.
- Menerapkan SRE memastikan bahwa keamanan bukan sekadar fitur tambahan (seperti menambahkan tombol login di akhir), melainkan komponen inti yang tertanam dalam arsitektur sistem.

Tahapan 1: Elicitation

Proses menggali dan mengumpulkan informasi untuk memahami apa saja yang perlu dilindungi.

- **Identifikasi Aset:** Menentukan aset berharga apa saja yang ada di dalam sistem (misalnya: data pribadi pengguna, kunci enkripsi, atau integritas transaksi keuangan).
- **Identifikasi Pelaku & Ancaman:** Memetakan siapa saja potensi penyerang (*attacker*) dan motivasi mereka.
- **Analisis Regulasi:** Menentukan standar hukum atau kepatuhan yang mengikat sistem tersebut (seperti UU Perlindungan Data Pribadi/PDP, ISO 27001, atau PCI-DSS).

Tahapan 2: Analysis & Negotiation

Menganalisis kebutuhan yang sudah dikumpulkan untuk menemukan konflik dan menentukan prioritas.

- **Misuse Cases / Abuse Cases:** Jika *Use Case* menggambarkan bagaimana pengguna sah menggunakan sistem, *Abuse Case* menggambarkan bagaimana penyerang mengeksploitasi sistem.
- **Resolusi Konflik:** Seringkali ada konflik antara *keamanan* dan *kenyamanan pengguna (usability)* atau biaya. Di tahap ini, tim pengembang, tim keamanan, dan pemangku kepentingan (*stakeholders*) bernegosiasi untuk menemukan titik temu yang dapat diterima (toleransi risiko).

Tahapan 3: Specification

Mendokumentasikan kebutuhan keamanan ke dalam format yang jelas, formal, dan dapat diukur agar bisa diimplementasikan oleh pengembang. Kebutuhan ini biasanya dibagi menjadi dua:

- **Kebutuhan Fungsional Keamanan:** Fitur keamanan yang langsung terlihat oleh pengguna (misalnya: "Sistem harus mengunci akun setelah 3 kali salah memasukkan password" atau "Sistem harus mengirimkan OTP melalui SMS").
- **Kebutuhan Non-Fungsional Keamanan:** Karakteristik kualitas sistem terkait keamanan (misalnya: "Semua data sensitif yang disimpan dalam database harus dienkripsi menggunakan algoritma AES-256").

Tahapan 4: Validation & Verification

Memastikan bahwa spesifikasi kebutuhan keamanan yang telah ditulis sudah benar, lengkap, konsisten, dan benar-benar menjawab risiko yang diidentifikasi.

- Apakah semua ancaman yang dipetakan sudah memiliki mitigasinya di dalam dokumen kebutuhan?
- Apakah kebutuhan tersebut realistis untuk diuji (*testable*) pada fase pengujian nanti?

Metode Kebijakan SRE yang Populer

Dalam dunia akademik dan industri, terdapat beberapa metode terstruktur yang digunakan untuk menjalankan SRE, di antaranya:

- **SQUARE (Security Quality Requirements Engineering):** Dikembangkan oleh Carnegie Mellon University (CMU), metode ini memiliki 9 langkah terstruktur untuk mendefinisikan, mengategorikan, dan memprioritaskan kebutuhan keamanan sistem.
- **CLASP (Comprehensive Lightweight Application Security Process):** Pendekatan berbasis aktivitas yang membantu organisasi mengintegrasikan praktik keamanan ke dalam proses manajemen kebutuhan yang sudah ada.
- **Tropos/Secure Tropos:** Pendekatan berbasis agen yang berfokus pada pemodelan kepercayaan (*trust*), delegasi kekuasaan, dan keamanan sejak fase analisis kebutuhan awal.

Tantangan dalam Menerapkan SRE

- 1. Kurangnya Kesadaran Keamanan:** Pemangku kepentingan sering kali lebih fokus pada fitur bisnis yang menghasilkan keuntungan langsung daripada aspek keamanan yang tidak terlihat di permukaan.
- 2. Kebutuhan yang Sering Berubah:** Pada metodologi modern seperti Agile, kebutuhan sistem berubah sangat cepat, sehingga menyelaraskan dokumen keamanan secara dinamis menjadi tantangan tersendiri.
- 3. Kompleksitas Teknis:** Menyeimbangkan antara keamanan yang ketat dengan performa sistem serta pengalaman pengguna yang mulus memerlukan keahlian mendalam.

Misuse and Abuse Cases and Attach Patterns

Misuse Cases, Abuse Cases, dan Attack Patterns

Dalam tahapan *Security Requirements Engineering* (SRE), untuk memahami bagaimana sistem harus bertahan, kita harus memahami bagaimana penyerang berpikir. Tiga konsep utama yang digunakan untuk memodelkan perspektif penyerang ini adalah **Misuse Cases**, **Abuse Cases**, dan **Attack Patterns**.

Misuse Cases (Kasus Salah Guna)

- **Definisi:** Deskripsi tentang aktivitas yang dilakukan oleh aktor (bisa penyerang sengaja, atau pengguna sah yang melakukan kesalahan) yang menghasilkan dampak negatif bagi sistem, tanpa memandang apakah ada niat jahat atau tidak.
- **Fokus:** Menekankan pada kegagalan sistem atau dampak buruk yang terjadi ketika batasan sistem dilanggar.
- **Contoh:**
 - *Sengaja:* Seorang mantan karyawan menggunakan akun lamanya yang belum dinonaktifkan untuk mengunduh data perusahaan.
 - *Tidak Sengaja:* Seorang dosen tidak sengaja mengunggah file nilai mahasiswa ke direktori publik yang bisa diakses siapa saja karena antarmuka aplikasi yang membingungkan.

Abuse Cases (Kasus Penyalahgunaan)

- **Definisi:** Deskripsi lengkap tentang interaksi antara sistem dengan **aktor penyerang (attacker)** yang memiliki niat jahat (*malicious intent*), di mana penyerang tersebut secara sadar mencoba merusak, mencuri, atau memanipulasi sistem.
- **Fokus:** Menekankan pada karakteristik penyerang, tujuan mereka, dan strategi serangan.
- **Contoh:**
 - Seorang peretas (*hacker*) secara sengaja membanjiri server aplikasi ujian *online* dengan jutaan *request* palsu (serangan DDoS) agar server *down* saat ujian berlangsung.

Attack Patterns (Pola Serangan)

Attack patterns adalah dokumentasi dari taktik, teknik, dan prosedur yang biasa digunakan oleh penyerang yang telah diklasifikasikan agar dapat digunakan kembali oleh tim pengembang untuk mendesain pertahanan. Basis data attack patterns yang paling terkenal secara global adalah **CAPEC (Common Attack Pattern Enumeration and Classification)** oleh MITRE.

Attack Patterns (Pola Serangan)

Beberapa contoh pola serangan adalah:

- Injection Patterns (Pola Injeksi)
- Spoofing & Impersonation (Pemalsuan Identitas)
- Resource Depletion (Pengurasan Sumber Daya)
- Abuse of Functionality (Penyalahgunaan Fungsi Logika)

Attack Patterns (Pola Serangan)

Injection Patterns (Pola Injeksi)

Penyerang memanfaatkan kolom input aplikasi yang tidak divalidasi dengan benar untuk memasukkan perintah atau skrip berbahaya ke dalam sistem backend.

- **SQL Injection (SQLi):** Memasukkan perintah SQL ke dalam form login untuk melewati proses autentikasi tanpa password.
- **Cross-Site Scripting (XSS):** Menyisipkan skrip JavaScript berbahaya ke dalam kolom komentar aplikasi web, yang kemudian akan dieksekusi di browser pengguna lain yang melihat komentar tersebut.

Attack Patterns (Pola Serangan)

Spoofing & Impersonation (Pemalsuan Identitas)

Penyerang berpura-pura menjadi entitas atau pengguna lain yang sah untuk mendapatkan hak akses.

- **Session Hijacking:** Mencuri token sesi (*session cookie*) milik pengguna lain yang sedang aktif untuk mengambil alih akun mereka tanpa perlu tahu password-nya.
- **Credential Stuffing:** Menggunakan bot secara otomatis untuk mencoba jutaan kombinasi username dan password yang bocor dari situs lain, berharap pengguna menggunakan password yang sama di aplikasi Anda.

Attack Patterns (Pola Serangan)

Resource Depletion (Pengurasan Sumber Daya)

Penyerang mencoba membuat aplikasi tidak dapat diakses (*unavailable*) dengan menghabiskan kapasitas komputasi server.

- **Denial of Service (DoS):** Mengirimkan beban komputasi yang sangat berat (seperti melakukan query database berskala besar berulang-ulang dalam satu waktu) hingga server kehabisan memori atau CPU.

Attack Patterns (Pola Serangan)

Abuse of Functionality (Penyalahgunaan Fungsi Logika)

Penyerang tidak merusak kode, melainkan memanipulasi alur logika bisnis (*business logic*) yang sah dari aplikasi untuk keuntungan mereka.

- **Contoh:** Pada aplikasi e-commerce, penyerang mengubah parameter harga produk menjadi Rp1,- menggunakan alat *intercept* (seperti Burp Suite) sesaat sebelum menekan tombol "Bayar", dan sistem menerima transaksi tersebut karena validasi harga hanya dilakukan di sisi *client* (front-end).

Security Risk Analysis

Security Risk Analysis

- **Security Risk Analysis (Analisis Risiko Keamanan)** adalah sebuah proses terstruktur untuk mengidentifikasi, menilai, dan mengevaluasi potensi risiko keamanan yang dapat mengancam aset berharga suatu sistem atau organisasi (seperti data sensitif, perangkat lunak, infrastruktur, hingga reputasi).
- Tujuan utama dari analisis ini bukanlah untuk menghilangkan semua risiko secara total (karena itu hampir mustahil dan sangat mahal), melainkan untuk **memahami risiko** yang ada sehingga organisasi dapat mengambil keputusan yang tepat untuk mengelola dan memitigasi risiko tersebut hingga ke tingkat yang dapat ditoleransi (*acceptable level of risk*).

Formula Dasar Resiko

Dalam dunia keamanan informasi, risiko (*Risk*) umumnya dihitung dan dipahami melalui hubungan antara tiga komponen utama: **Aset**, **Ancaman**, dan **Celah Keamanan**.

$$Risk = Threat \times Vulnerability \times Asset Value$$

- **Asset (Aset):** Apa yang ingin dilindungi? (Contoh: Database nilai mahasiswa, source code aplikasi, data transaksi).
- **Threat (Ancaman):** Siapa atau apa yang bisa merusaknya? (Contoh: Hacker, malware, orang dalam yang berniat jahat, atau bencana alam).
- **Vulnerability (Celah Keamanan):** Apa kelemahan pada sistem yang bisa dieksploitasi? (Contoh: Port server yang terbuka, tidak ada enkripsi data, atau password admin yang lemah).
- Jika salah satu elemen di sisi kanan bernilai nol (misalnya, ada ancaman besar tetapi sistem sama sekali tidak memiliki celah keamanan terkait), maka risikonya menjadi sangat rendah atau nol.

Tahapan dalam Security Risk Analysis

Proses analisis risiko biasanya mengikuti langkah-langkah sistematis berikut:

1. Identifikasi Aset dan Penilaian Nilai Aset
2. Identifikasi Ancaman (*Threat Identification*)
3. Identifikasi Celah Keamanan (*Vulnerability Identification*)
4. Penilaian Risiko (*Risk Assessment*)
5. Penentuan Strategi Mitigasi (*Risk Treatment*)

Metode Pendekatan Analisis Risiko

Ada dua pendekatan utama yang sering digunakan untuk mengukur risiko:

1. Analisis Kualitatif (Qualitative Analysis):

1. Pendekatan yang paling sering digunakan karena lebih praktis.
2. Menggunakan skala deskriptif seperti **High, Medium, Low** atau angka 1–5 berdasarkan penilaian subjektif para ahli (*expert judgment*).
3. *Output-nya* berupa matriks risiko (*Risk Matrix*).

2. Analisis Kuantitatif (Quantitative Analysis):

1. Pendekatan berbasis data numerik dan finansial yang presisi.
2. Menghitung perkiraan kerugian nyata dalam mata uang.
3. Menggunakan metrik seperti **ALE (Annualized Loss Expectancy)** untuk menghitung berapa kerugian finansial yang diprediksi terjadi akibat risiko tersebut dalam satu tahun.

Urgensi Security Risk Analysis

- Tanpa *Security Risk Analysis*, sebuah tim atau organisasi akan buta dalam mengalokasikan sumber daya keamanan mereka. Mereka mungkin menghabiskan anggaran besar untuk mengamankan aset yang kurang penting, sementara aset yang menjadi jantung utama bisnis justru dibiarkan tanpa proteksi yang memadai.
- Analisis risiko memastikan bahwa **investasi keamanan siber yang dikeluarkan tepat sasaran** dan efisien.

Threat Modeling, Prioritization, and Mitigation

Threat Modeling, Prioritization, and Mitigation

- **Threat Modeling, Prioritization, and Mitigation** adalah triad proses yang saling berkesinambungan dalam arsitektur keamanan perangkat lunak. Ketiganya merupakan langkah konkret untuk menerjemahkan hasil analisis risiko menjadi tindakan pertahanan yang nyata.
- Jika diibaratkan membangun sebuah rumah, *Threat Modeling* adalah cara kita memetakan dari mana saja pencuri bisa masuk, *Prioritization* adalah menentukan pintu mana yang paling rawan dibobol, dan *Mitigation* adalah tindakan memasang gembok atau kamera pengawas pada pintu tersebut.

Threat Modeling

- **Threat Modeling** adalah aktivitas terstruktur untuk mengidentifikasi potensi ancaman, agen penyerang, dan kerentanan yang mungkin mengeksploitasi sistem, dilakukan *sebelum* atau *selama* kode program ditulis.
- Proses ini biasanya menjawab empat pertanyaan dasar (prinsip Adam Shostack):
 1. *Apa yang sedang kita bangun?* (Memetakan arsitektur sistem)
 2. *Apa saja hal buruk yang bisa terjadi?* (Mengidentifikasi ancaman)
 3. *Apa yang akan kita lakukan untuk mengatasinya?* (Mitigasi)
 4. *Apakah kita sudah melakukan pekerjaan dengan baik?* (Validasi)

Threat Modeling: STRIDE

Salah satu kerangka kerja paling standar dalam *Threat Modeling* adalah **STRIDE** (dikembangkan oleh Microsoft), yang membagi ancaman perangkat lunak ke dalam 6 kategori:

- **Spoofing** (Pemalsuan Identitas): Penyerang berpura-pura menjadi pengguna atau sistem lain yang sah.
- **Tampering** (Manipulasi Data): Mengubah data secara ilegal saat disimpan atau dikirim (misal: mengubah parameter harga barang).
- **Repudiation** (Penyangkalan): Pengguna melakukan aksi jahat tetapi sistem tidak punya bukti (*log*) untuk membuktikannya.
- **Information Disclosure** (Kebocoran Informasi): Data sensitif terekspos ke pihak yang tidak berwenang.
- **Denial of Service** (DoS): Membuat aplikasi lumpuh atau tidak dapat diakses oleh pengguna sah.
- **Elevation of Privilege** (Peningkatan Hak Akses): Pengguna biasa berhasil mendapatkan hak akses sebagai Admin.

Prioritization

Setelah melakukan *Threat Modeling*, sebuah tim biasanya akan menemukan puluhan hingga ratusan potensi ancaman. Tidak semua ancaman bisa atau perlu diperbaiki sekaligus karena keterbatasan waktu, anggaran, dan sumber daya manusia. Di sinilah **Prioritization** berperan untuk menentukan ancaman mana yang harus ditangani terlebih dahulu.

Prioritization: DREAD

Untuk memberikan nilai prioritas yang objektif, tim sering menggunakan model skor **DREAD**:

- **Damage Potential** (Potensi Kerusakan): Seberapa parah kerusakan jika serangan ini berhasil?
- **Reproducibility** (Kemudahan Mereproduksi): Seberapa mudah serangan ini diulangi oleh penyerang lain?
- **Exploitability** (Kemudahan Eksploitasi): Seberapa mudah celah ini ditemukan dan dieksploitasi (apakah butuh keahlian tinggi atau bisa menggunakan *tools* otomatis)?
- **Affected Users** (Pengguna Terdampak): Berapa banyak pengguna yang akan terkena dampaknya?
- **Discoverability** (Kemudahan Penemuan): Seberapa mudah penyerang menemukan celah ini di dalam sistem?

Setiap poin diberi nilai (misal 1–3 atau 1–10). Ancaman dengan total skor tertinggi akan masuk ke dalam daftar prioritas utama (**High Priority**) untuk segera dimitigasi.

Mitigation

- **Mitigation** adalah langkah-langkah teknis atau prosedural yang diambil untuk meredam, mengisolasi, atau menghilangkan ancaman yang telah diprioritaskan tadi.
- Dalam rekayasa perangkat lunak, mitigasi dilakukan dengan memasang *Security Controls* (Kendali Keamanan).

Mitigation

Kategori STRIDE	Taktik Mitigasi Teknis	Contoh Nyata pada Aplikasi
Spoofing	Autentikasi yang Kuat	Menerapkan Multi-Factor Authentication (MFA) atau enkripsi token JWT yang aman.
Tampering	Integritas Data & Validasi	Menggunakan fungsi Hashing (seperti SHA-256), tanda tangan digital, dan melakukan <i>input validation</i> yang ketat di sisi server.
Repudiation	Audit Trail & Logging	Membuat sistem pencatatan aktivitas (<i>secure logging</i>) yang mencatat setiap aksi krusial pengguna, di mana log tersebut bersifat <i>read-only</i> .
Information Disclosure	Enkripsi Data	Menggunakan protokol HTTPS (TLS) untuk data yang berpindah, dan enkripsi AES-256 untuk data yang disimpan di database.
Denial of Service	Pengelolaan Sumber Daya	Menerapkan <i>Rate Limiting</i> (membatasi jumlah request per menit dari satu IP) dan menggunakan layanan proteksi DDoS (seperti Cloudflare).
Elevation of Privilege	Otorisasi Ketat	Menerapkan <i>Role-Based Access Control</i> (RBAC) dan memastikan pengecekan hak akses dilakukan di setiap <i>endpoint API</i> .

Ancaman dan Keamanan

Threat

Spoofing
Tampering
Repudiation
Information disclosure
Denial of service
Elevation of privilege

Security Property

Authentication
Integrity
Nonrepudiation
Confidentiality
Availability
Authorization

Kategori Ancaman menurut DFD

Element	Spoofing	Tampering	Repudiation	Information Disclosure	Denial of Service	Elevation of Privilege
Data flows		X		X	X	
Data stores		X		X	X	
Processes	X	X	X	X	X	X
External entity	X		X			